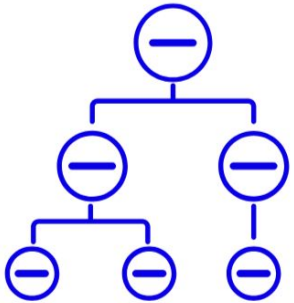
A Framework For Benchmarking Graph Based Artificial Intelligence

Thesis Defence
Kent O'Sullivan

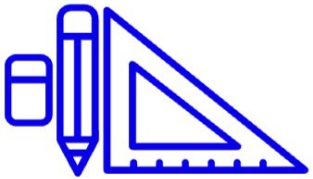
Advisor: Prof Bill Regli
17 Apr 24

A Framework for GraphAI Benchmarking

The core contributions of this research are:



GraphAI Taxonomy



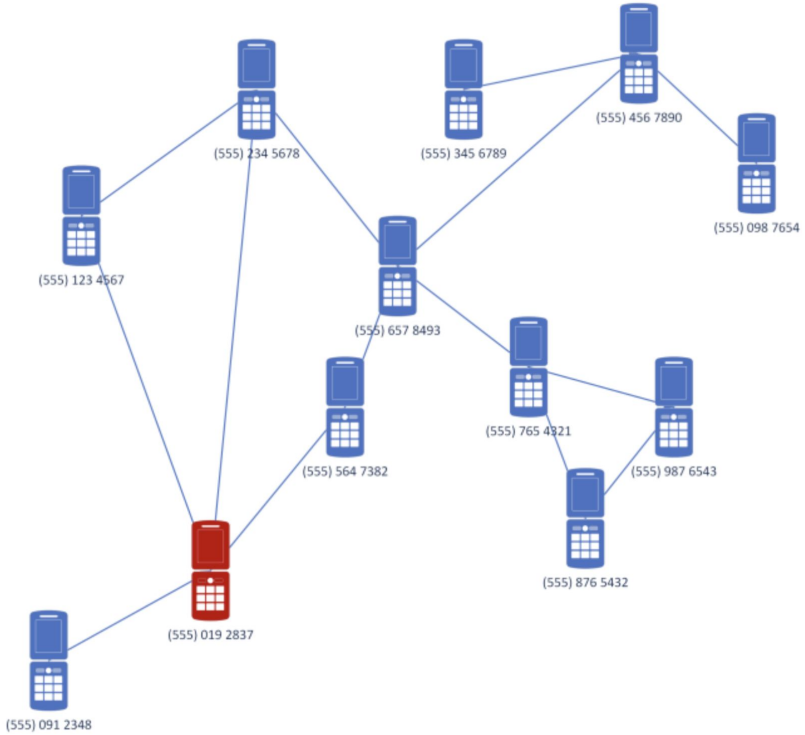
Benchmark Methodology



Population Segmentation Benchmark



Effectiveness/ Efficiency Analysis

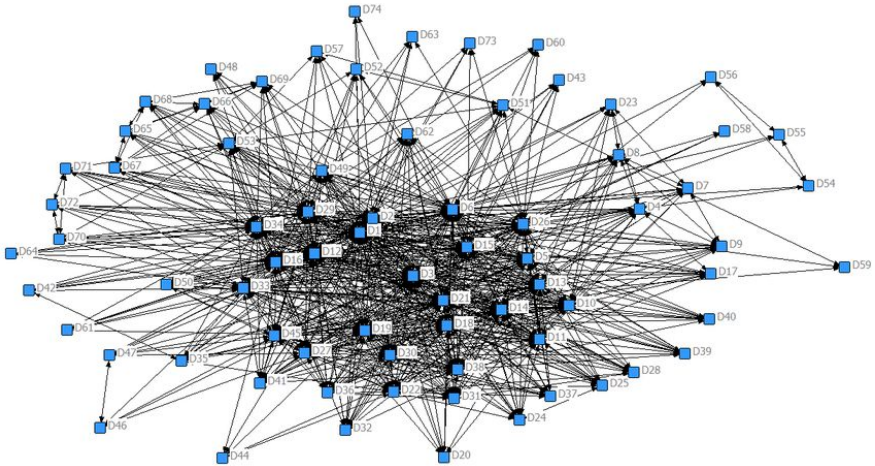


This is **a graph**

Phones are **vertices** and calls **edges**

Graphs (& GraphAI) **solve many problems**

Image Sources
https://www.researchgate.net/figure/Communication-network-diagram-between-construction-units-concerned_fig2_261094786
https://journalofbigdata.springeropen.com/articles/10.1186/s40537-019-0264-6



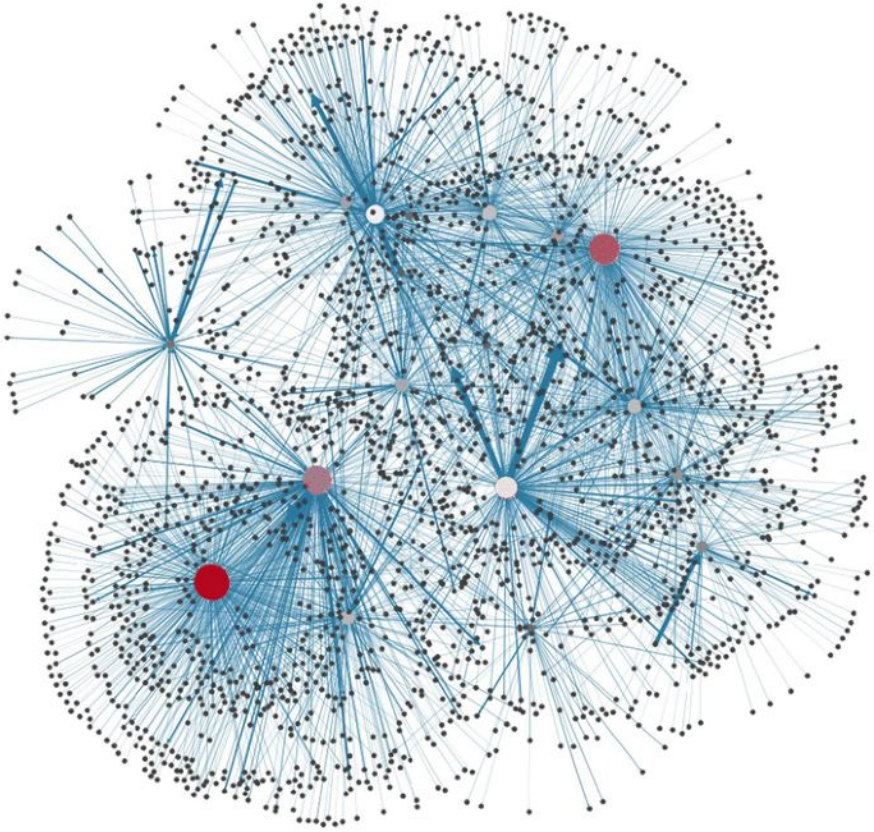
This is **a graph**

Phones are **vertices** and calls **edges**

Graphs (& GraphAI) **solve many problems**

But. Graphs get **big** and complex

Image Sources
https://www.researchgate.net/figure/Communication-network-diagram-between-construction-units-concerned_fig2_261094786
https://journalofbigdata.springeropen.com/articles/10.1186/s40537-019-0264-6



This is **a graph**

Phones are **vertices** and calls **edges**

Graphs (& GraphAI) **solve many problems**

But. Graphs get **big** and complex

… really big

Solving problems gets **slow** on big graphs

We can **speed up** problem solving by:

- Distributing
- Parallelizing
- Compressing
- Upgrading hardware
- Adding accelerators
- Approximating

Image Sources
https://www.researchgate.net/figure/Communication-network-diagram-between-construction-units-concerned_fig2_261094786
https://journalofbigdata.springeropen.com/articles/10.1186/s40537-019-0264-6

What if there was a way to compare the impact each improvement has on a system that solves problems with GraphAI?

So that we can consistently improve GraphAI performance while justifying our investments empirically.

For example, determining the cost effectiveness of adding hardware accelerators versus distributing the work to reduce processing time.

Further, a repeatable process can be easily extended to all problems in the GraphAI taxonomy.

Image Source: https://www.youtube.com/watch2

Agenda

1. Introduction
    a. Problem Definition
    b. Prior Work
    c. Novelty
    d. Impact

2. GraphAI Benchmarking
    a. GraphAI Taxonomy
    b. GraphAI Challenges
    c. The GraphAI Benchmarking Framework
    d. The GraphAI Benchmark Problem Development Methodology

3. Case Study
    a. Developing a Population Segmentation Problem Benchmark
    b. Findings

4. Future Work

INTRODUCTION

Introduction: Problem Definition

This thesis aims to determine *if*, *how* and *when* to speed up problem solving on linked data given a set of operating *constraints*.

How*:* to effectively combine the representations, hardware accelerators and algorithms.

If & When*:* it is appropriate to exploit acceleration to speed up the time to solution.

Constraints: on the *efficiency, effectiveness* and/or *cost* of a solution.

Introduction: Example Acceleration

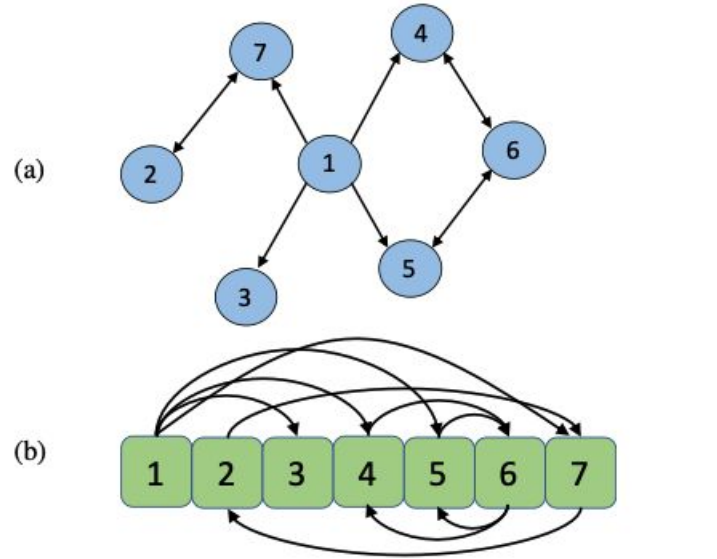
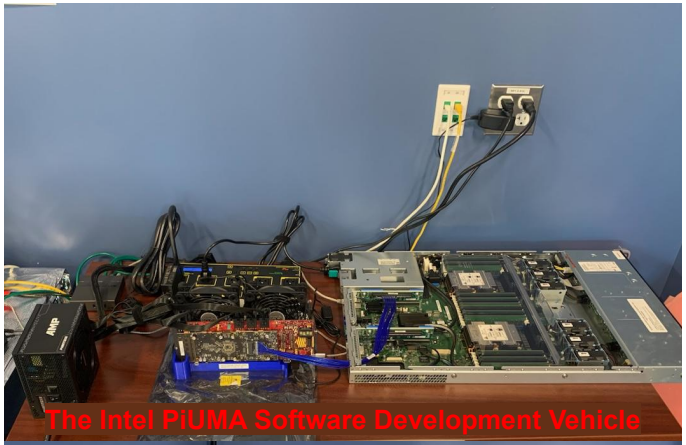
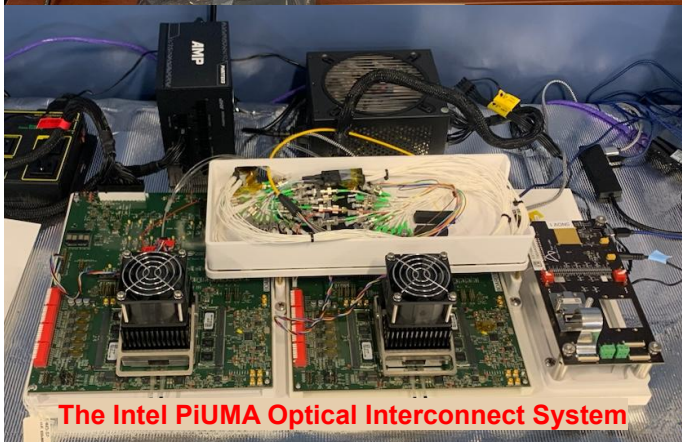


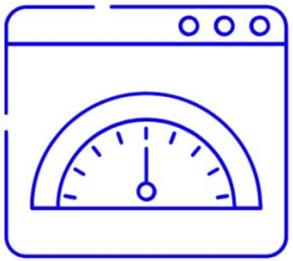
Fig. 1. (a) A sparse graph with directed edges. (b) Memory access patterns observed when moving data along the edges of (a).





Prototype Graph Hardware Accelerators (right) aim to address some of the causes of slowdown – but how well?

Introduction: Prior Work



`Black-Box' Benchmarks

Race to maximize *effectiveness* or *efficiency.*

Focus on the *kernels*, and *datasets* rather than *problems.*

Lack variety of *shapes* & support for *dynamic* graphs



`White-Box' Workload Analyses

Monitor internal system state to find *bottlenecks*

Require invasive system *profiling* & code *instrumentation*

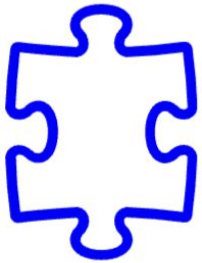
Lack *scalability*

Introduction: Novelty

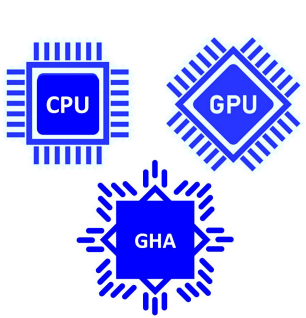
Our GraphAI benchmark framework builds on prior work by offering:



Balanced Benchmark



Problem Centric



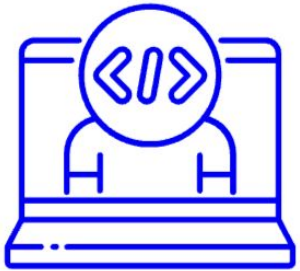
Accelerator Agnostic



Efficiency vs Effectiveness

Introduction: Impact

The GraphAI Benchmark has broad utility to three key groups:



Analysts



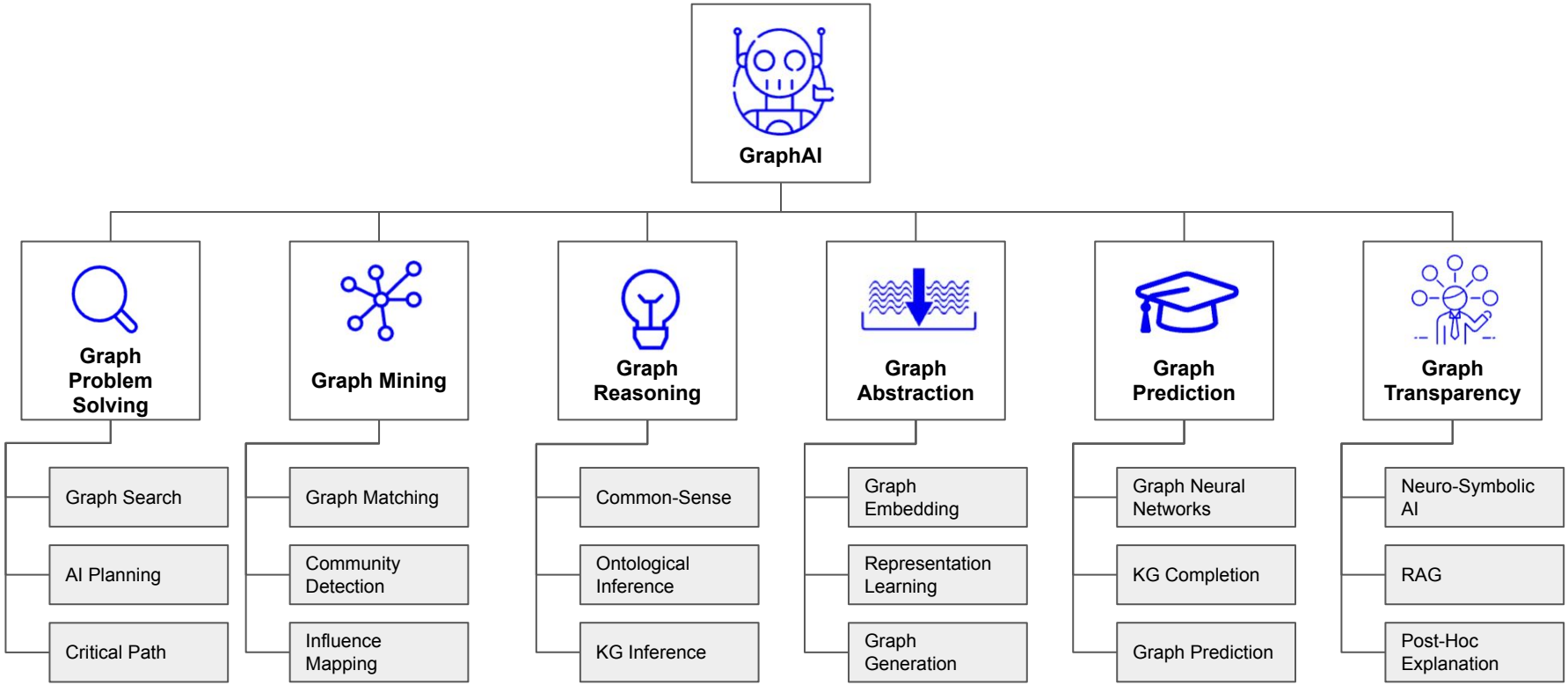
Engineers



Researchers

The GraphAI Benchmark Framework

Our GraphAI Taxonomy



Related Work: https://chenxuhao.github.io/graphAI.html

GraphAI Benchmarking Challenges

Evaluation:

If, How, When & Constraints?

User:

Effectiveness or Efficiency?

‘Racing’ systems or invasive analysis?

Real World:

Imposes Constraints

Accelerators are specialized

System:Problem ratio

GraphAI:

GraphAI problems behave differently

Acceleration is dependent on graph processing

Graph:

Graph Processing is Data Dependent

Lack of Spatial and Temporal Locality

No standard abstraction, representation or primitives

A `Grey-Box' approach to GraphAI Benchmarking

BLACK BOX

- Inputs known.
- System details unknown.
- Implementation unknown.
- Outputs evaluated.

GREY BOX

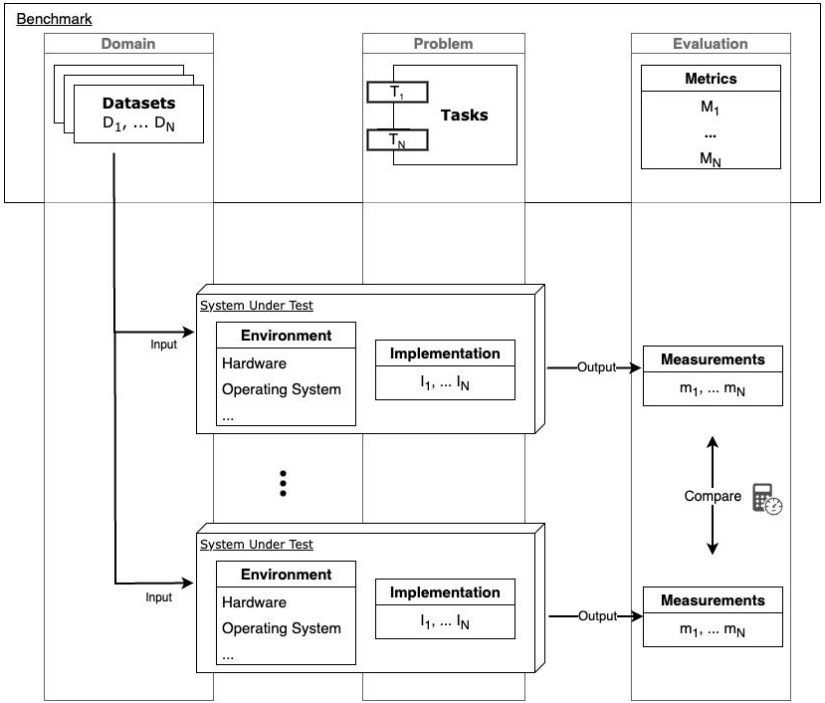
- Inputs known & designed to stimulate bottlenecks.
- System detail unknown.
- Implementation unknown.
- Outputs evaluated and analyzed to infer system and implementation detail.

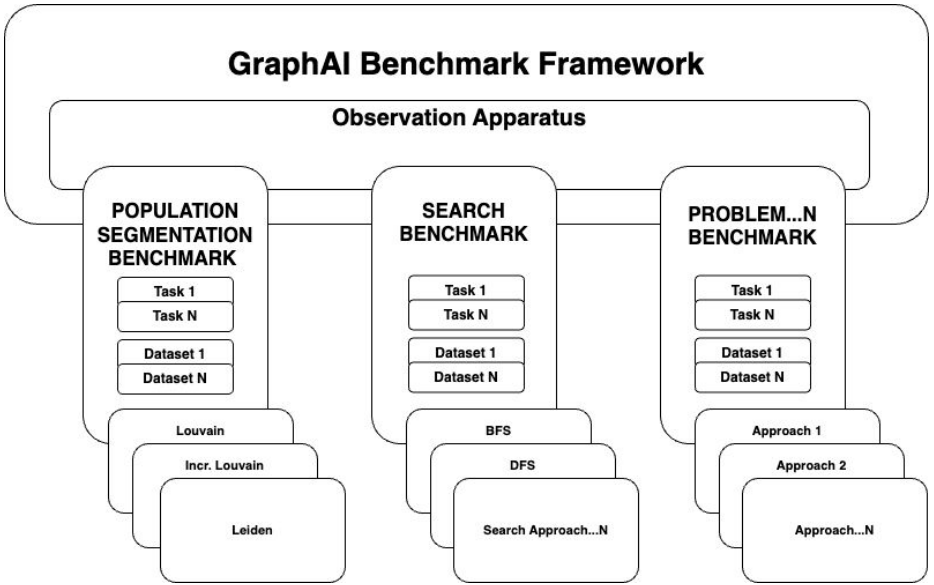
WHITE BOX

- Inputs known.
- System detail known and instrumented.
- Implementation known and instrumented.
- Outputs rarely evaluated.

The GraphAI Benchmark Framework

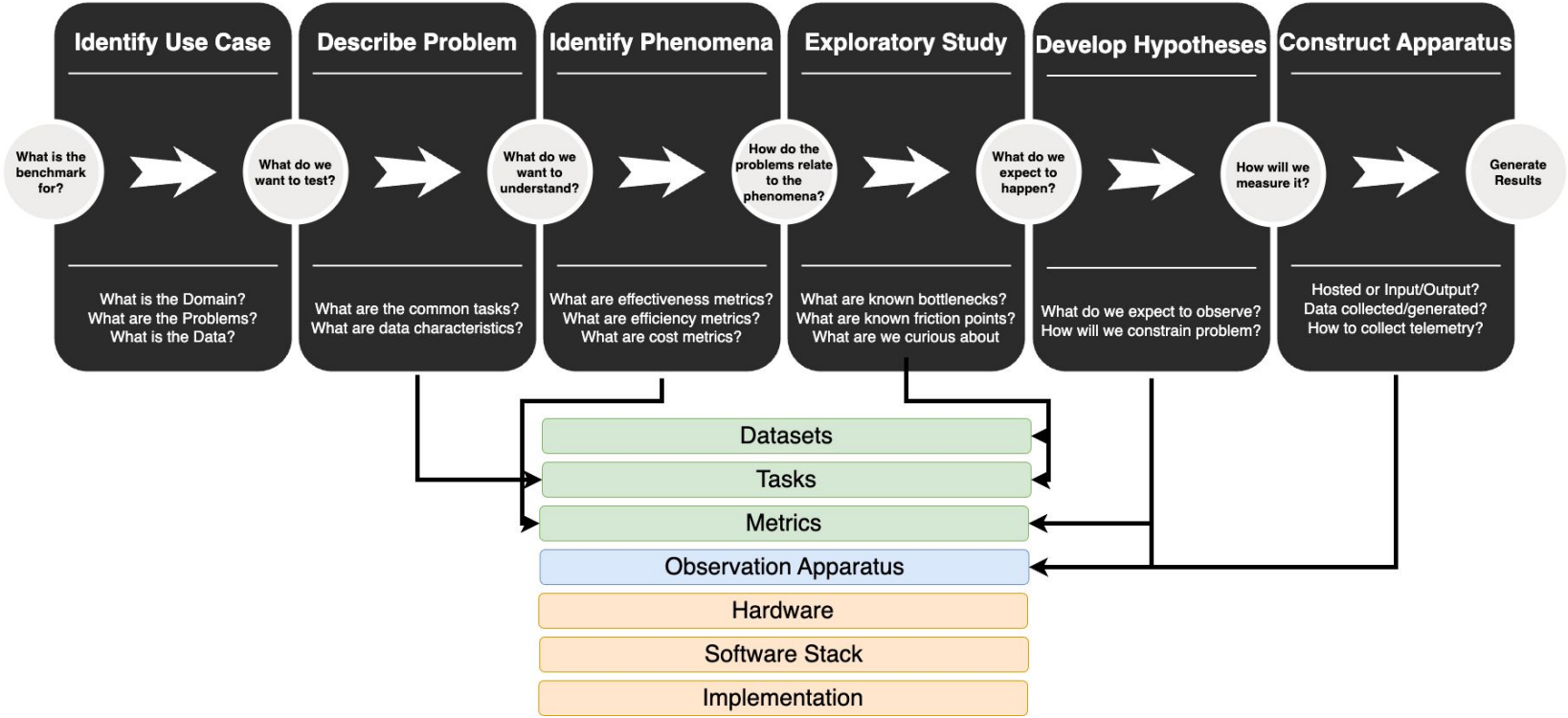
A typical benchmark framework (left) underpins our GraphAI Framework (right)





Graph AI Problem Development Methodology

We develop a formal empirical method to develop GraphAI benchmark problems:

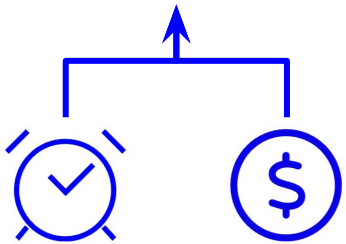


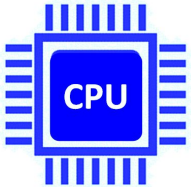
CASE STUDY

Applying the GraphAI Benchmark to Population Segmentation

Use Case

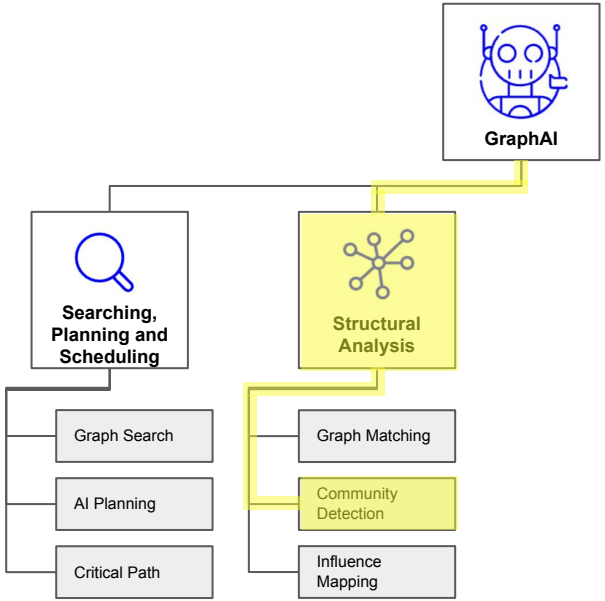






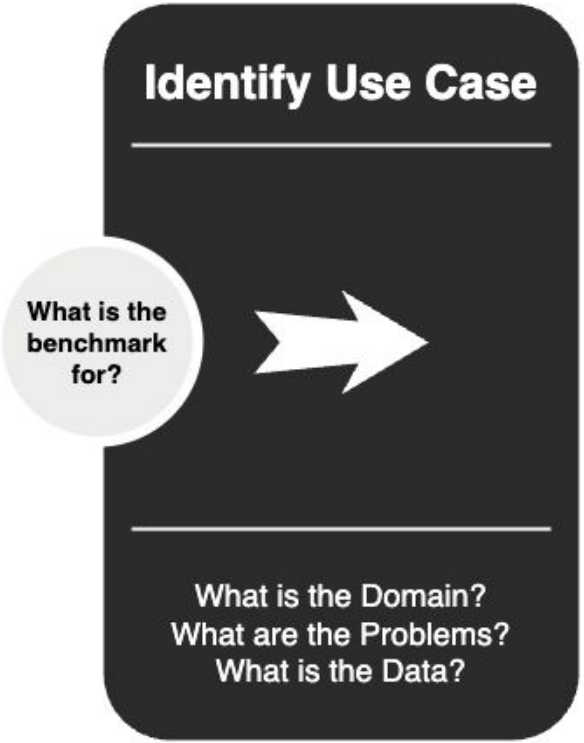






An engineer architecting a new system needs to identify whether GPUs or specialized GHAs are a worthwhile investment for their GraphAI workloads given time and cost constraints.

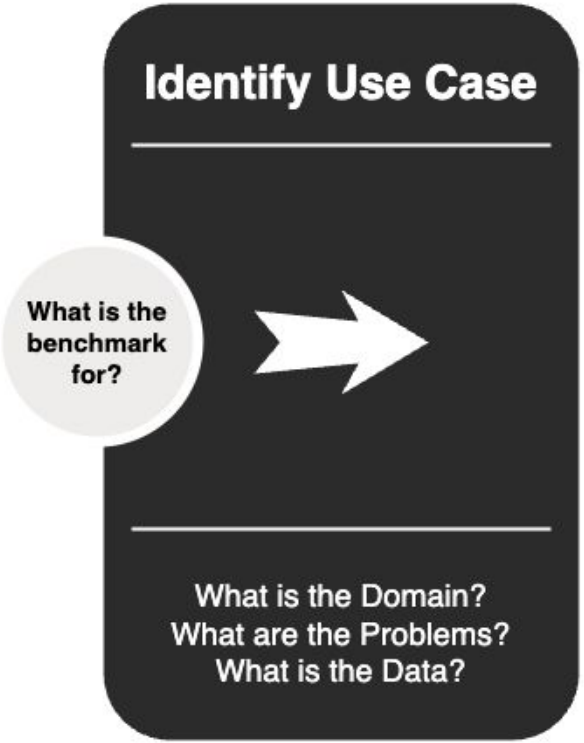
Step 1: Identify the Use Case

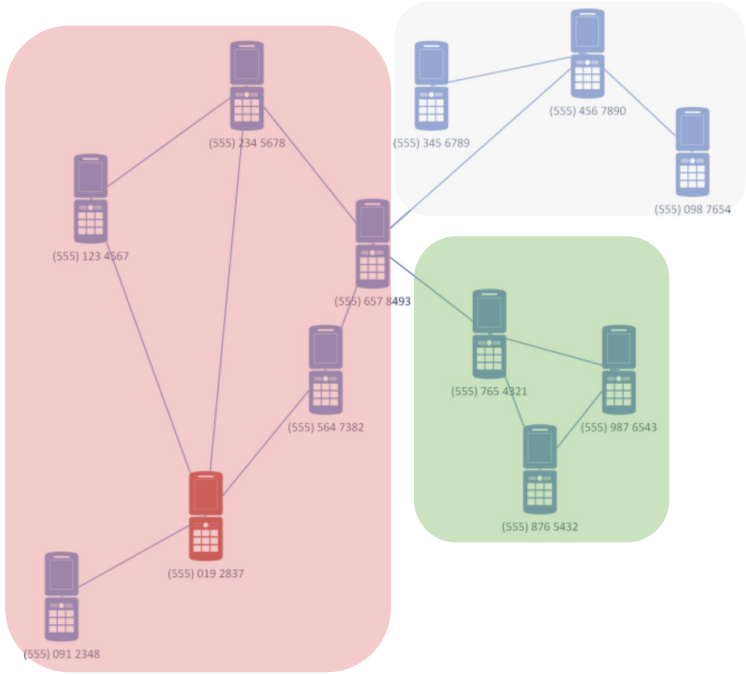


AI Problem	I&S Problem	Input Data	GraphAI Approach	Constraints
GPS	Link Analysis [31]	Knowledge Graph	Reachability	Heterogeneity
	Navigation	Spatial graph	Path Planning	SWaP; Complexity
GM	HVT Identification	Multi-Source Graphs	Influence Mapping	Heterogeneity
	Co-Traveller Detection [30, 32–35]	Spatio-Temporal Data	Clustering	Temporal
	POI Identification [6, 36]	Location Data	Spatial Pattern Matching	Complexity
	Demographic Segmentation [37–39]	Communication Logs, Social Media	Community Detection	Temporal
	Counter-Threat Finance [40–44]	Transaction Records, Blockchain	Subgraph Matching	Temporal
GR	Kill-Chain Analysis [45–48]	Event Logs	Logical Inference	Completeness
	All-Source Fusion [30, 49, 50]	Streaming Knowledge Graph	Entity Resoultion	Temporal
GP	Anomaly Detection [51, 52]	Streaming Knowledge Graph	Classification	Temporal
	Link Prediction [53–56]	Streaming Knowledge Graph	Completion	Temporal

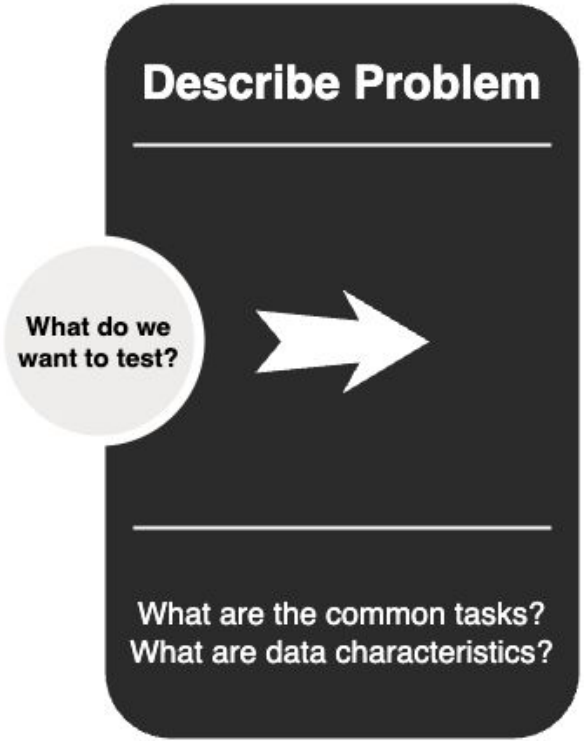
A survey of Intelligence and Security use cases leads us to select *Population Segmentation*

Step 1: Identify the Use Case





Population Segmentation: Given a population, how can we divide it so that the most ‘like’ members are together?



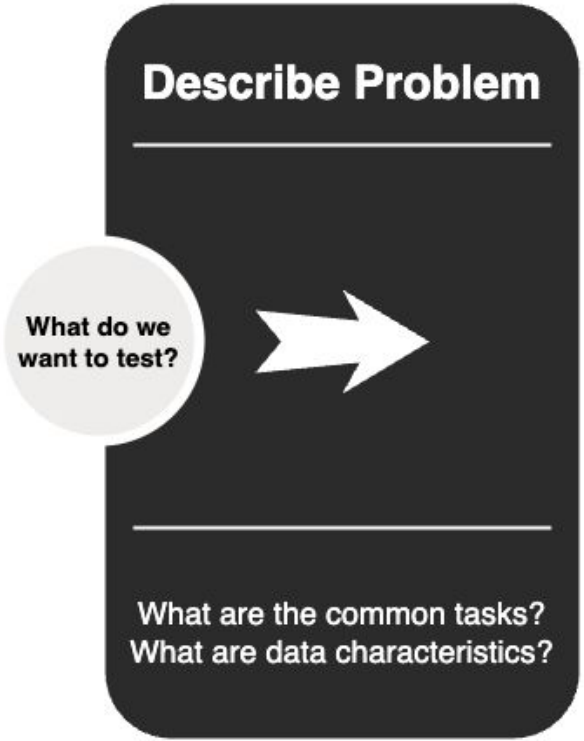
Step 2: Describe the Problem

Tasks:

- Graph Partitioning
- Graph Clustering
- Community Detection
- Dynamic Detection

Dataset Characteristics:

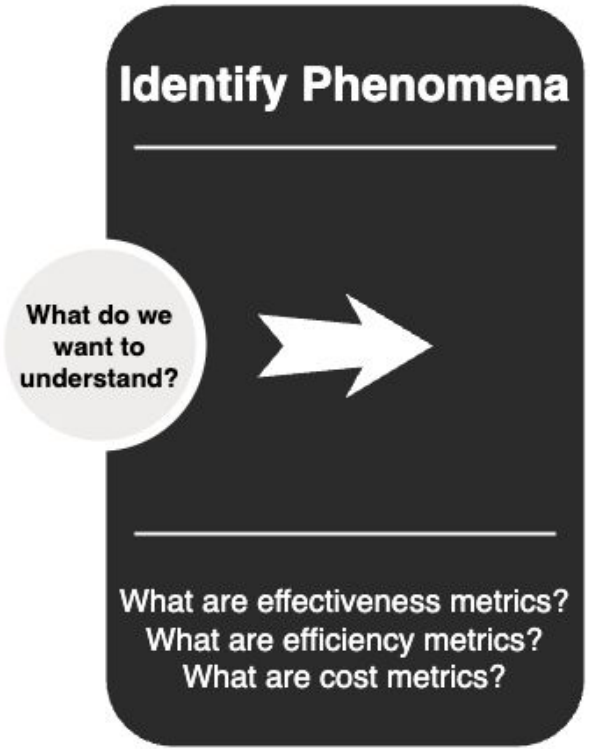
- Scale-Free
- ‘Clique-y’ vs ‘Fuzzy’
- Scale quadratic in Vertex Count
- Attributed and Weighted



Step 2: Describe the Problem

Problem	Output Format
[1] Who is in the same group as `(x)`?	List of Vertex IDs
[2] Is `(x)` likely to be know `(y)`?	Tuple of (Y/N, Confidence)
[3] Count the vertices in the largest and smallest disjoint segments.	Tuple of (small_size,big_size,[small],[big])
[4] `(x)`, `(y)` and `(z)` are in communities `(a)`, `(b)` and `(c)`. Who else is in `(c)`?	List of Vertex IDs
[5] Who shares a segment with both `(a)` with property `(1)` and `(b)` with property `(2)`?	List of Vertex IDs
[6] Which segment has the most vertices with property `(3)`?	Tuple of (Group_name, [Vertex IDs])
[7] What are the segments and who is in each?	List of Lists of Vertex IDs
[8] There are `(n)` segments, which segment does each vertex belong to?	List of Lists of Vertex IDs

Select *problem-centric* tasks and expected outputs for population segmentation.



Step 3: Identify the Phenomena

Given the problem:

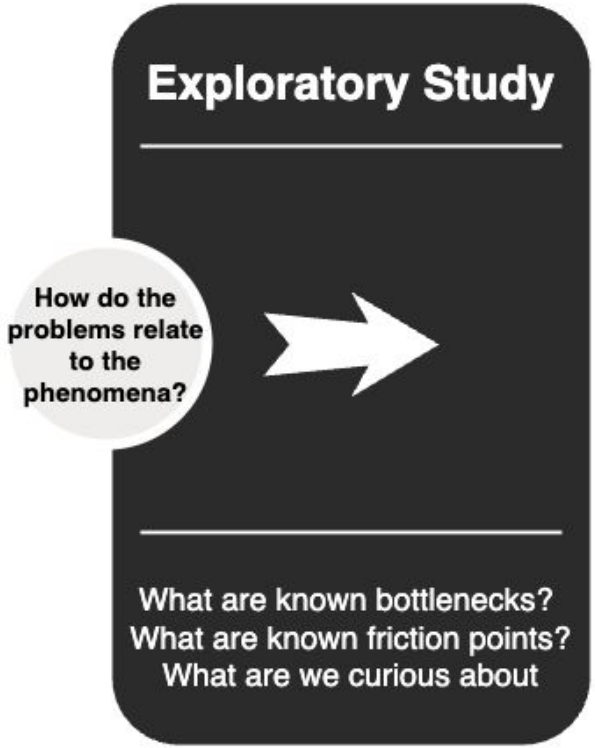
[7] What are the segments and who is in each?

We are interested in the phenomena:

What is the interaction between *effectiveness* and *efficiency* for population segmentation approaches?

Effectiveness Metric: Normalized Mutual Information of segments

Efficiency Metric: Batch Latency

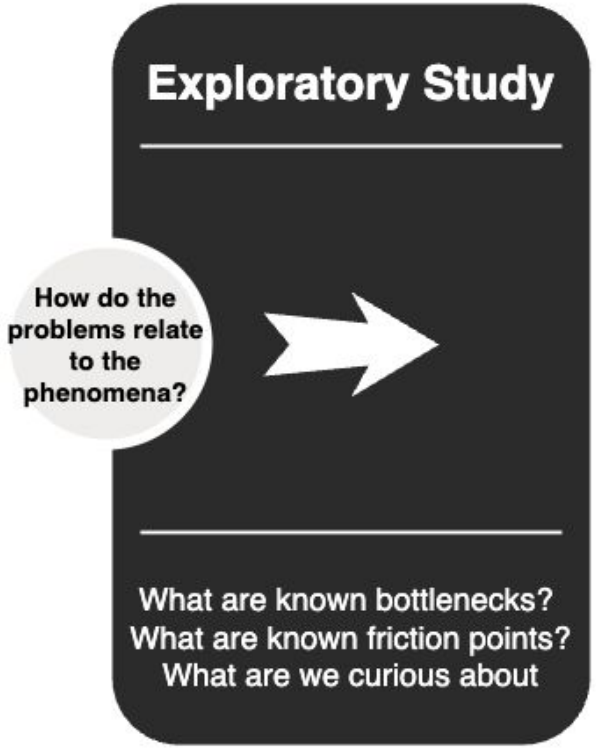


Step 4: Conduct Exploratory Study

Family	Algorithm	Efficiency Bottleneck	Effectiveness Botleneck
Graph Partitioning	Kernighan-Lin [153]	$\mathcal{O}(n^3)$	Dense; Small communities; Unknown K
	Extremal Mod. Optimization [154]	$\mathcal{O}(n^2 \log^2 n)$	Fuzzy
	Spectral Partitioning$_{base}$ [155]	$\mathcal{O}((m+n)n)$	Dense
	Spectral Partitioning$_{cl}$ [155]	$\mathcal{O}(n^3)$	Dense
	FEC [156]	$\mathcal{O}(cl(n+m))$	Fuzzy; Many communities; No Attributes
	Potts-Fuzzy [157]	$\mathcal{O}(m+q)$	$L_{in} \gg L_{out}$; Unknown C
Graph Clustering	Spectral Clustering [158]	$\mathcal{O}(n^3)$ [159]	Unknown C
	Girvan-Newman [7]	$\mathcal{O}(n^2 m)$	Fuzzy; Dense
	Hierarchical Agg. [160]	$\mathcal{O}(md \log n)$	Dense; Large C, Small C; Fuzzy
	Hierarchical Div. [149]	$\mathcal{O}((m+n)mn^2)$	Dense; Fuzzy
	Spectral [159]	$\mathcal{O}(nc^2 e)$	Skewed Community Distro; Unknown C
	Louvain [161]	$\mathcal{O}(n^2)$	Many Clique-Bridge Vertices
	Leiden [162]	$\mathcal{O}(n^2)$	Fuzzy
Graph Prediction	DNR [163]	$\mathcal{O}((nh+h^2)+(incd))$	Fuzzy; Unknown C
	Com-E [164]	$\mathcal{O}(n+m)$	Dense; Fuzzy
	M-NMF [165]	$\mathcal{O}(n(m^2 n))$	Dense; Fuzzy; Many C; Unknown C
Dynamic Graphs	Instant Optimal [148, 166]	$\mathcal{O}(T((n^2)+(c^2)))$	Fuzzy; Many C
	Temporal tradeoff [148, 167]	$\mathcal{O}(T(md \log n))$	Many changes per T
	Cross-Time [148, 168]	$\mathcal{O}(3^{\frac{d}{3}} 2^2 mnt)$	Dense

n *= number of vertices*
m *= number of edges*
c *= number of communities*
i *= number of iterations*
t *= number of timesteps*
d *= degeneracy of graph*

A literature review of prior benchmarks, workload analyses & accelerator designs explores the problem



Step 4: Conduct Exploratory Study

Bottlenecks

Efficiency degrades most with size of graph

Effectiveness degrades most with fuzziness of communities

Friction Points

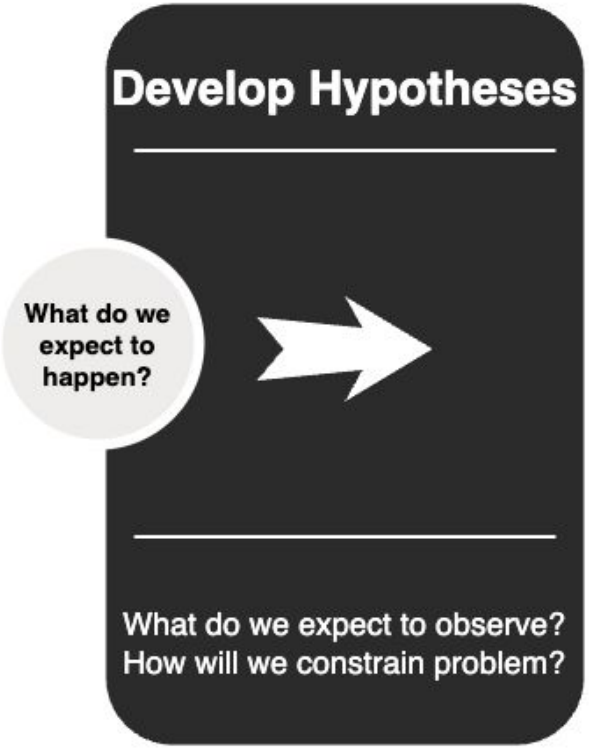
When the number of segments is not known

When a vertex can belong to many segments

When graphs are dynamic

Curiosities and Opportunities

Use of incremental and approximate approaches



Step 5: Develop Hypotheses

↑ GPU and other Graph Hardware Accelerator (GHA) hardware configurations will have a longer ETL time relative to the same input data and implementation on the baseline system.

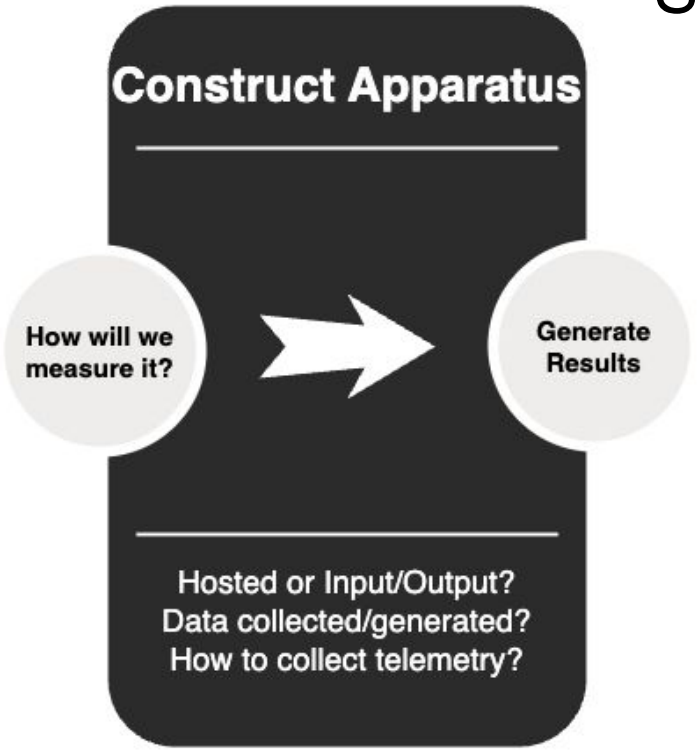
↑ GPU will have longer ETL time for scale-free input graphs than for mesh input graphs of the same size.

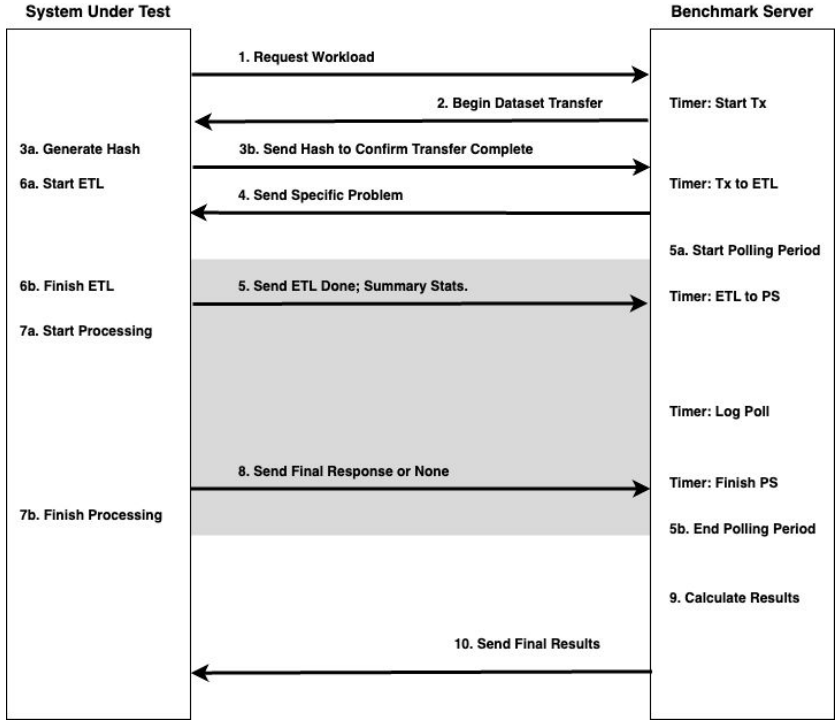
↑ The effectiveness of each workload is independent of the hardware configuration used.

= The BL of single-shot approaches will be equal to their ETL plus Ex Time for all inputs.

↑ The effectiveness of an iterative or incremental implementation will improve monotonically over time as it converges to an optimal state.

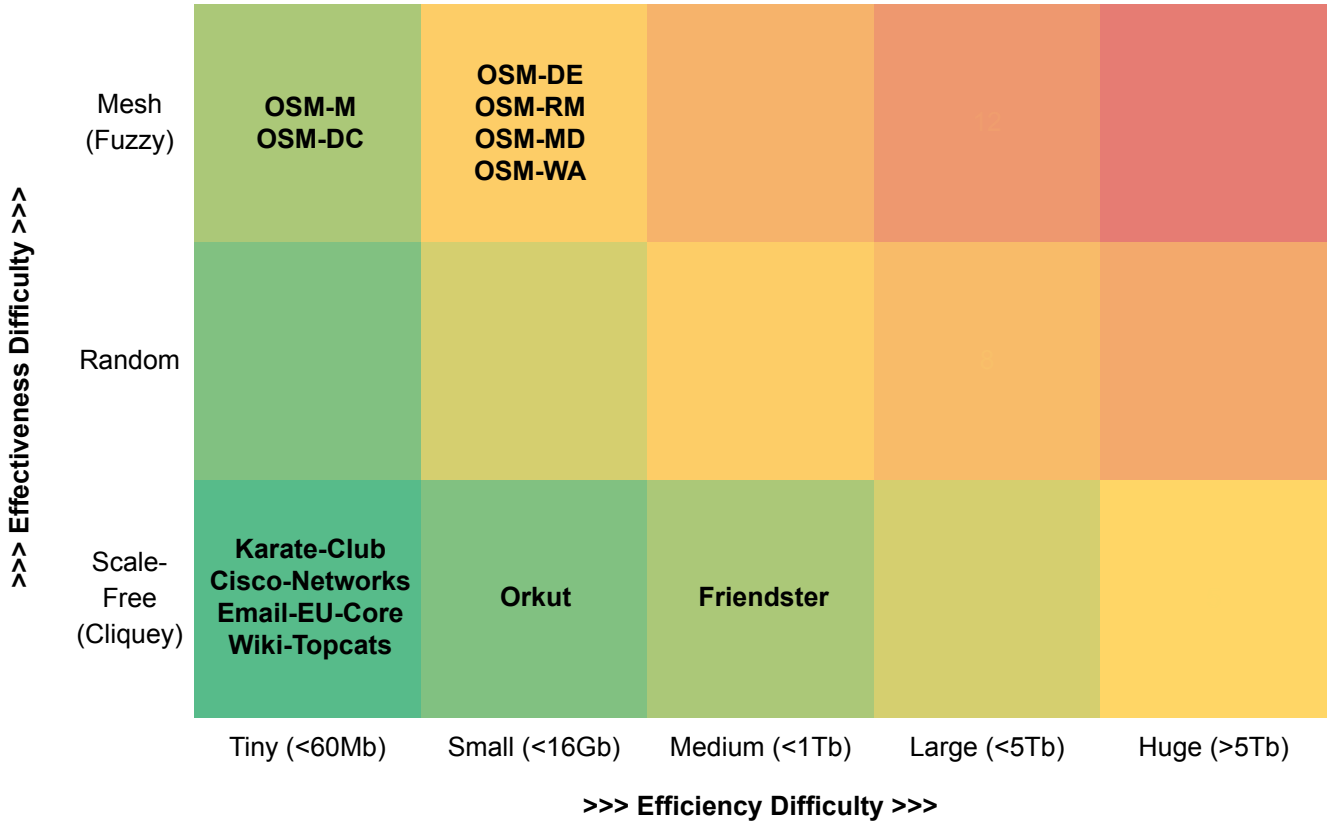
Step 6: Construct Apparatus



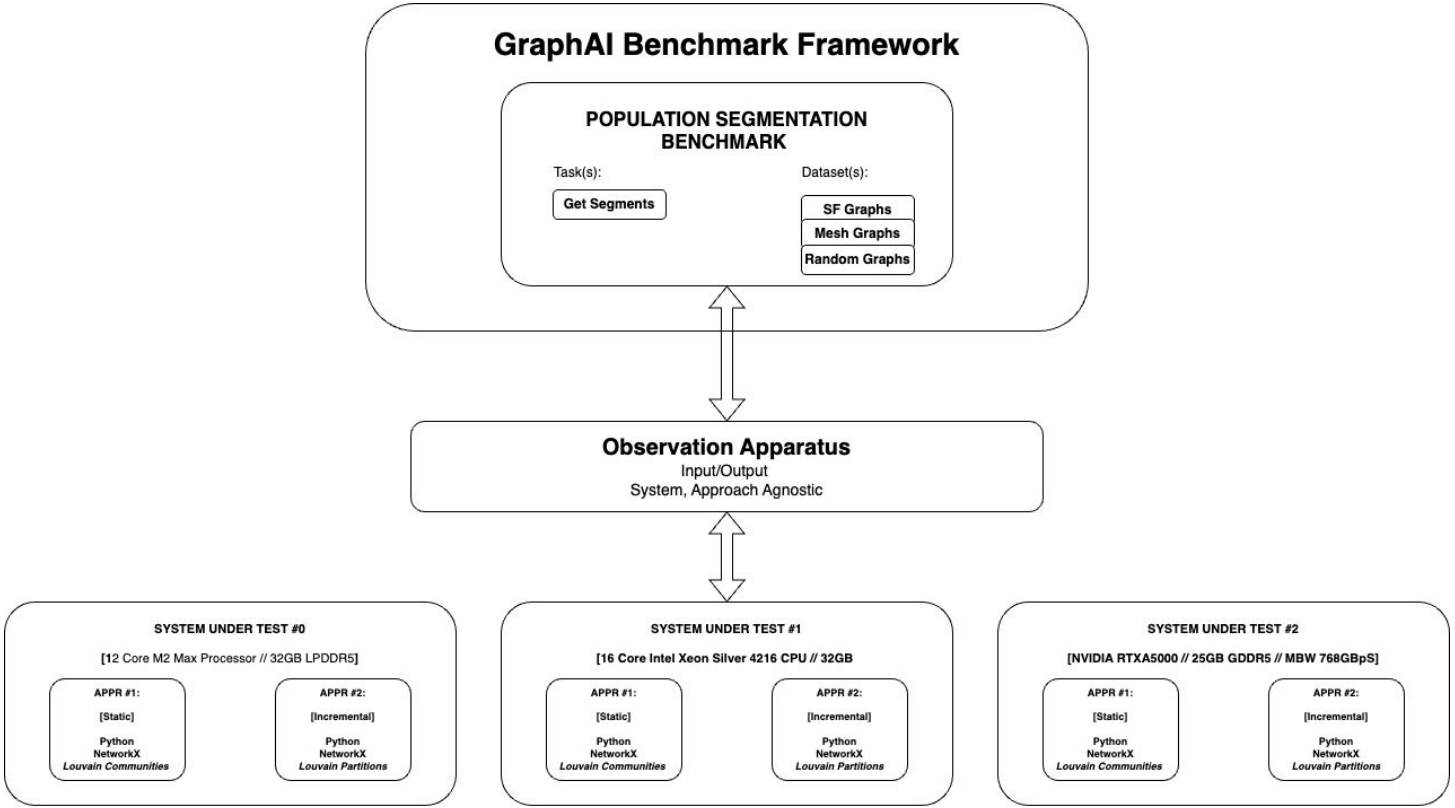


The apparatus is a system and approach-agnostic Input/Output model that measures effectiveness vs efficiency.

Collect Datasets

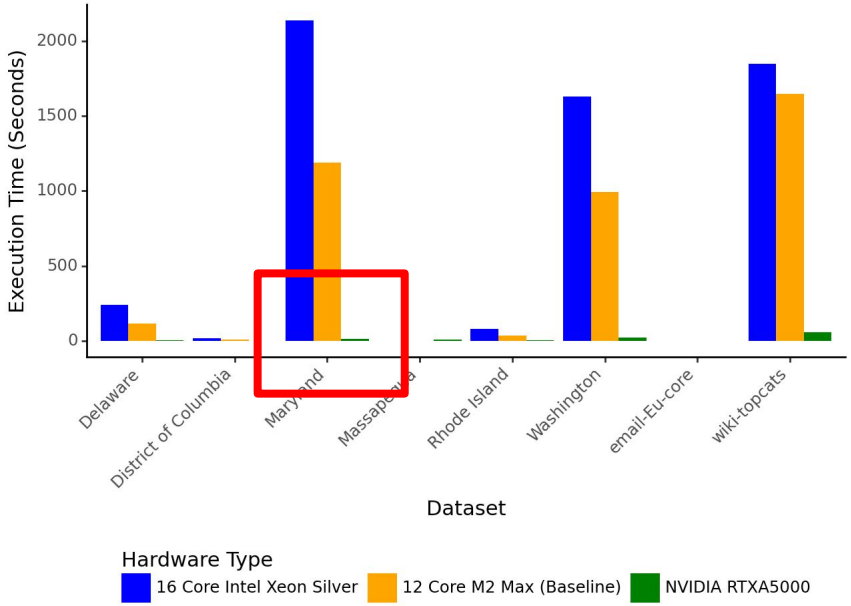


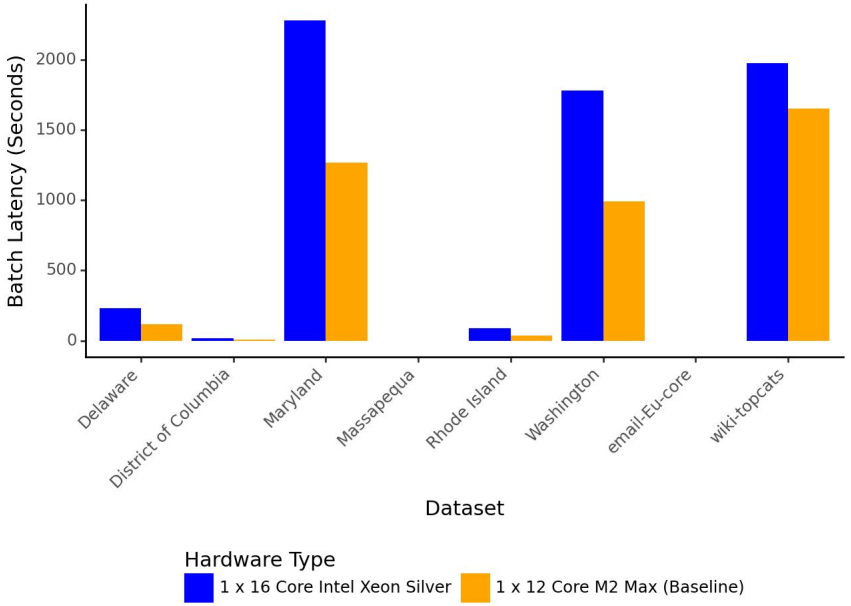
Experiment Setup



Setup for demonstration of the Population Segmentation benchmark.

Results - Execution Time

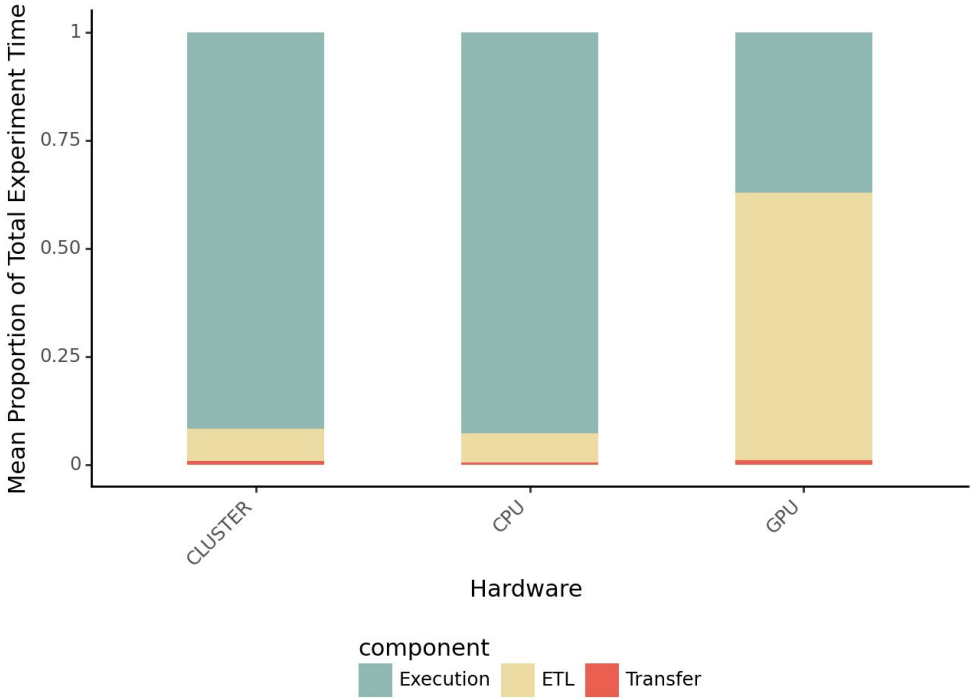




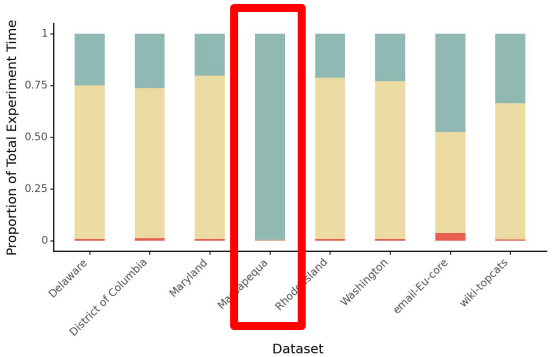
Using a GPU yields a up to a **172x speed-up** in execution time on MD Roads dataset; but does not support the incremental approach.

Batch Latency @1 roughly equals execution time – the ‘incremental’ approach is **not incremental in time.**

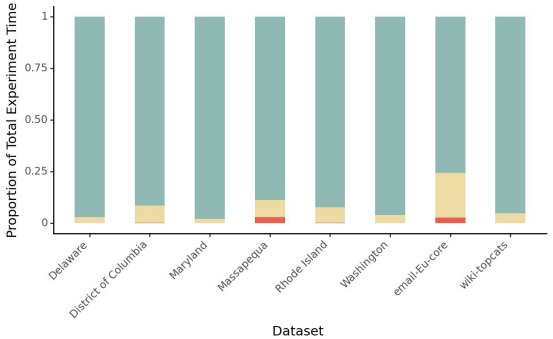
Results - Time Usage



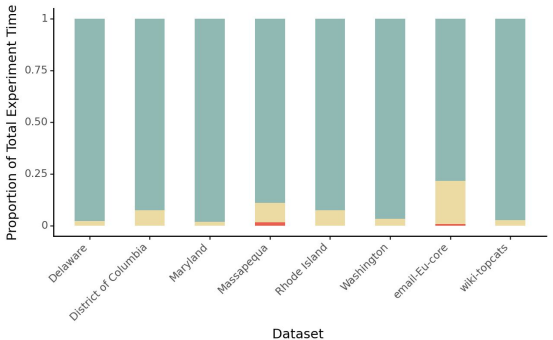
ETL is a much greater proportion of Processing time for GPU-based approaches than for CPU-based approaches. Massapequa may be below partition threshold.



GPU



Xeon

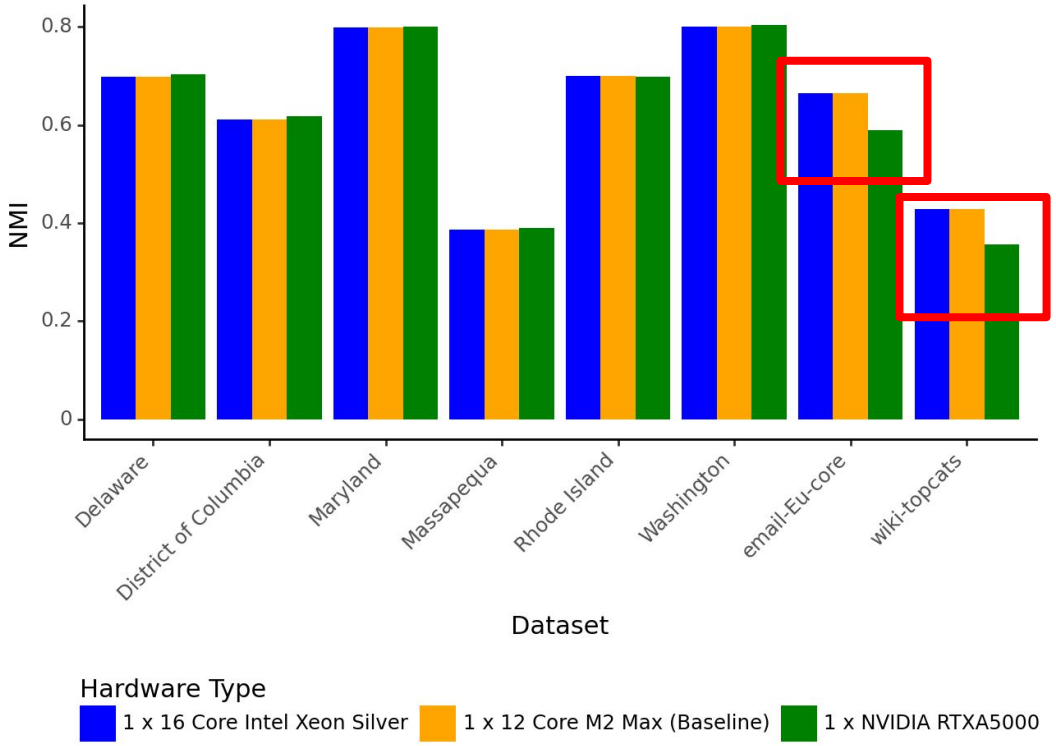


M2 Max

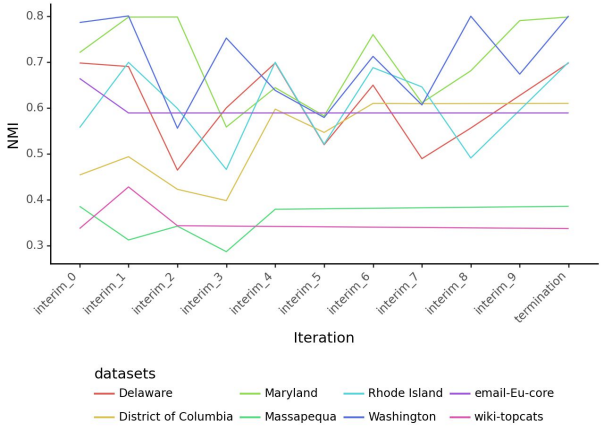
Results - Effectiveness of Static Approaches

Effectiveness degrades for GPU on scale-free datasets using the ‘static’ algorithm.

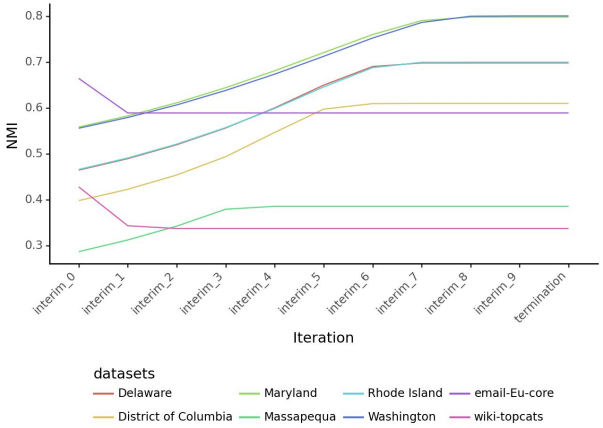
Likely due to initial partitioning scheme used by GPU.



Results - Effectiveness of Incremental Approaches (NMI)



M2 Max



Xeon Silver

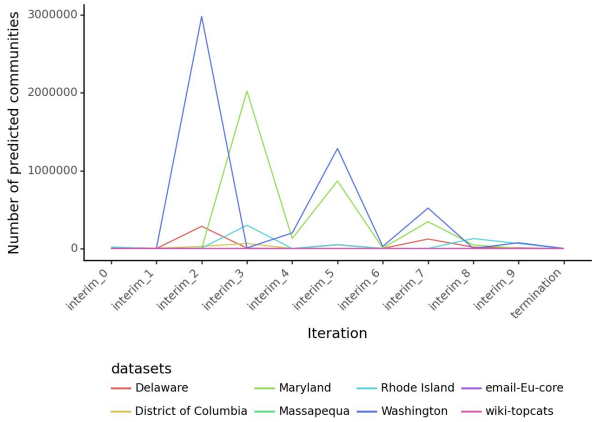
No Incremental Implementation for GPU

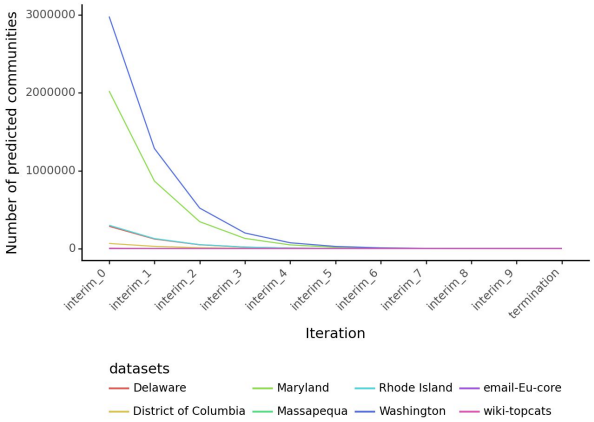
RTXA5000

The M2 Max implementation exhibits non-monotonic behavior in NMI improvement.

One could trade effectiveness for efficiency on the Xeon Silver.

Results - Effectiveness of Incremental Approaches (#Seg)





No Incremental Implementation for GPU

M2 Max

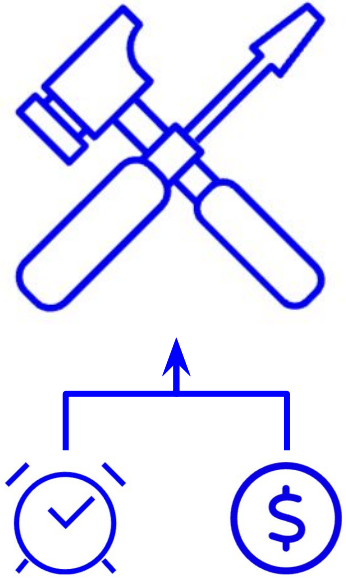
Xeon Silver

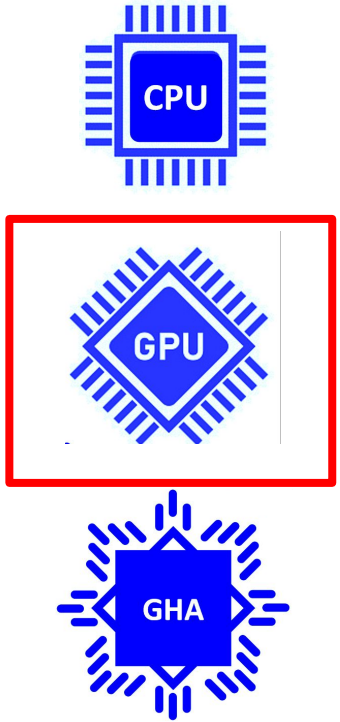
RTXA5000

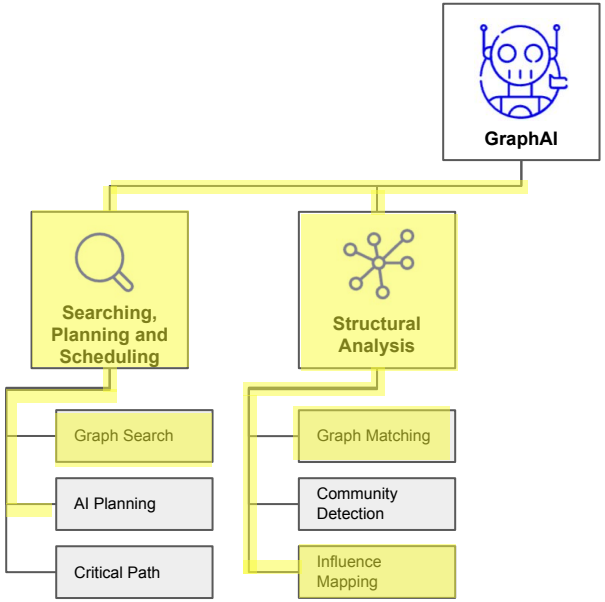
The M2 Max implementation exhibits non-monotonic behaviour in Segment size.

The ‘granularity’ of communities gets coarser over time on the Xeon Silver.

Decision:







Based on Population Segmentation alone, recommend spending ~$2500 on RTX A5000.
We **could** trade effectiveness for efficiency, but GPU will perform better these small datasets.

Full assessment requires repeating the process for each GraphAI Problem.

Future Work

Measuring Cost

- Currently measure *Effectiveness* and *Efficiency*
- *Power* is a good proxy for *cost*; but requires a hosted solution

Expanding Coverage to other GraphAI Problems

- Next: Isomorphic Subgraph Matching for Spatial Pattern Matching

Engineering

- API-Based model; hosted leaderboard; dynamic inputs

Maturity Model

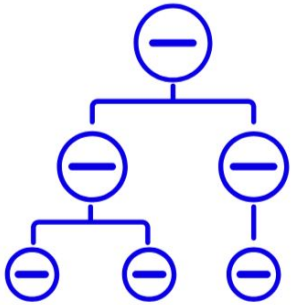
- Focus appears to gradually shift from effectiveness to efficiency as fields develop

Summary

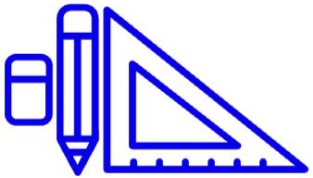
Problem:

To determine *how* and *when* to speed up problem solving on linked data given a set of operating *constraints*.

Contributions:



GraphAI Taxonomy



Benchmark Methodology



Population Segmentation Benchmark



Effectiveness/ Efficiency Analysis

Questions

If you have any questions, please reach out on:

osullik@umd.edu

+1 (202) 257 5565 [US Cell]

+61 (0) 431 133 963 [Aus Cell for Signal, Whatsapp etc]